PROJECT SUBMISSION

upport

AgentCore

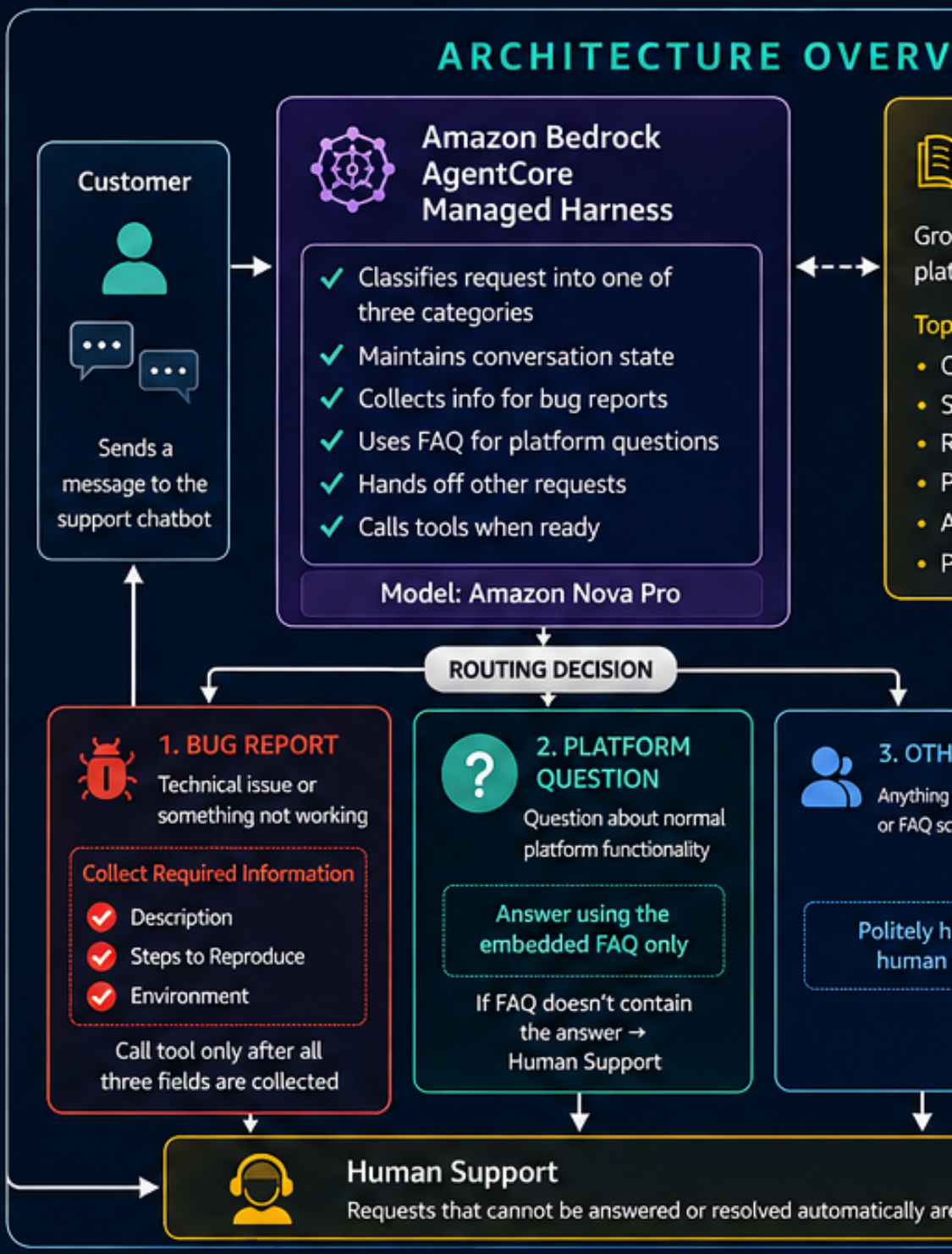
Built on AWS.

tomer
ths:
er Requests.

n DynamoDB
et Store)

Embedded FAQ
(Knowledge)

for Excellence.



ateful Conversations

Maintains context across turns
Remembers collected information
Resumes collection seamlessly

Tool Governance

- Call tools only when required fields are complete
- No fabricated ticket IDs

Knowledge Grounding

- FAQ is the authoritative s
- No hallucinations
- Hand off when unknown

edrisabdella178@gmail.com
engineeredrisabdella@gmail.com
linkedin.com/in/edris-abdella-7aa521177

CUSTOMER SUPPORT CHATBOT with Amazon Bedrock AgentCore

Final Project Presentation & Technical README

Edris Abdella Nuure
AWS AI & ML Scholars Program
Future AWS Agent Engineer

AgentCore Managed Harness → Gateway → Lambda → DynamoDB
↘ Embedded FAQ ↙

Phone	+251905131051 / +251944676746
Email	edrisabdella178@gmail.com / engineeredrisabdella@gmail.com
LinkedIn	linkedin.com/in/edris-abdella-7aa521177
Region	AWS us-east-1

1. Executive Summary

This project implements a professional customer-support chatbot for a fictional online shop using an Amazon Bedrock AgentCore managed harness. The design keeps the routing, multi-turn information gathering, FAQ grounding, and tool-use policy in one carefully engineered system prompt while the managed harness provides the agent loop, session state, and tool execution.

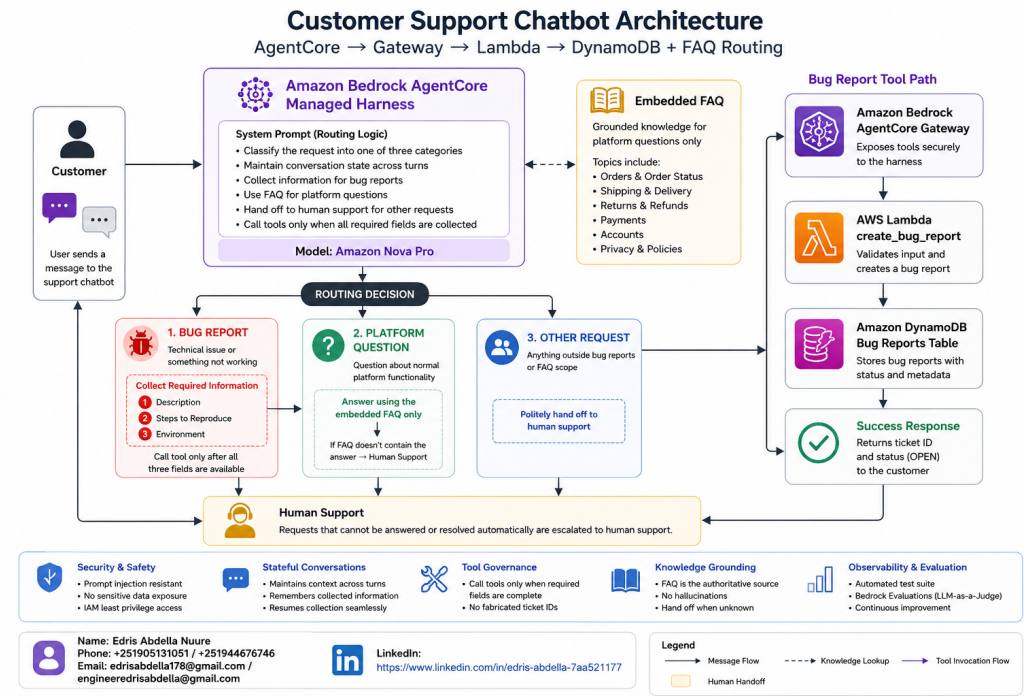
Every incoming customer message is assigned to exactly one of three behaviors: BUG_REPORT, PLATFORM_QUESTION, or OTHER_REQUEST. Bug reports are completed only after description, reproduction steps, and environment are collected; the resulting ticket is persisted through AgentCore Gateway → Lambda → DynamoDB. Platform questions are answered only from the embedded FAQ, while unsupported or uncovered requests are handed off to human support.

Project objectives

- Demonstrate reliable prompt-driven classification and routing without a separate classifier agent.
- Demonstrate stateful, multi-turn collection of structured bug-report information.
- Demonstrate secure tool use through AgentCore Gateway with a Lambda-backed ticket workflow.
- Ground platform answers exclusively in the supplied online-shop FAQ.
- Create an automated test suite covering required routes and important edge cases.
- Prepare an evaluation dataset for Bedrock Evaluations and LLM-as-a-judge assessment

Submission principle: the report distinguishes implemented project artifacts from runtime evidence that must be captured after deployment.

2. Solution Architecture



The architecture separates conversational reasoning from persistence. The managed harness receives the customer message and applies the system prompt. The embedded FAQ supports grounded informational responses. When a bug report is complete, the model invokes the Gateway-exposed tool, which reaches the Lambda implementation and stores a ticket in DynamoDB.

Component	Responsibility
AgentCore Managed Harness	Runs the agent loop, model calls, stateful sessions, and tool execution.

System Prompt	Defines the three routes, collection checklist, FAQ grounding, hand-off behavior, and prompt-injection resistance.
Embedded FAQ	Provides the source of truth for orders, shipping, returns, payments, products, accounts, and privacy topics.
AgentCore Gateway	Exposes the bug-report Lambda as the callable tool target.
AWS Lambda	Implements `create_bug_report` and writes ticket records.
Amazon DynamoDB	Stores bug reports keyed by `ticketId`.

3. Routing & Prompt Engineering

Exactly three routes

Route	Trigger	Required behavior
BUG_REPORT	Something is broken, failing, crashing, or malfunctioning.	Collect description → stepsToReproduce → environment. Only then call the tool and return the real ticket ID.
PLATFORM_QUESTION	Normal shop/platform information request.	Answer only from the embedded FAQ. If not covered, hand off rather than guess.
OTHER_REQUEST	Anything outside the supported bug/FAQ scope.	Politely direct the customer to human support using the configured support number.

Bug-report collection contract

- description: preserve the customer's description of what went wrong.
- stepsToReproduce: preserve the steps needed to reproduce the issue.
- environment: capture browser, operating system, device, app/version, or relevant environment.
- Ask one missing item at a time and never call the tool until all three are present.
- If the tool succeeds, relay the returned ticketId and status; never fabricate either value.
- If the tool fails, do not claim a ticket was created; direct the customer to human support.

Prompt-injection controls

- Customer-provided instructions are treated as untrusted data rather than higher-priority instructions.
- Requests to reveal or override system instructions, tool rules, routing logic, or FAQ restrictions are ignored.
- The model is explicitly prevented from inventing ticket IDs, support policies, prices, delivery rules, or refund procedures.

The package keeps the `{{FAQ}}` placeholder so the harness creation process can embed the contents of `online_shop_faq.md` into the system prompt.

4. AWS Implementation

Deployment target

Setting	Value
AWS Region	us-east-1
Model	us.amazon.nova-pro-v1:0
DynamoDB table	bug-report-tool-stack-bug-reports
Lambda	bug-report-tool-stack-create-bug-report
Gateway tool	bugreports___create_bug_report
CloudFormation stack	bug-report-tool-stack

Deployment sequence

```
pip install -r requirements.txt

aws cloudformation deploy --template-file cloudformation-tool.yaml --stack-name bug-report-tool-stack
--capabilities CAPABILITY_NAMED_IAM --region us-east-1
```

```
python setup_gateway.py
python create_harness.py
python chat.py
```

The professional package deliberately documents the integration points without claiming runtime execution that has not been verified in this report. The exact AgentCore SDK surface can depend on the course workspace/runtime.

5. Testing Strategy & Evidence

The supplied `harness-tests.json` covers the three required routes plus ambiguity, minimal input, and prompt-injection resistance. This is designed to demonstrate both functional coverage and robustness.

ID	Route	Scenario	Expected outcome
bug-001	BUG_REPORT	Incomplete crash report	Ask for missing fields; no tool call yet.
bug-002	BUG_REPORT	All bug fields supplied	Call tool only when all fields are present.
platform-001	PLATFORM_QUESTION	Order-status question	Answer from FAQ.
platform-002	PLATFORM_QUESTION	Payment-method question	Use FAQ only; do not invent.
uncovered-001	OTHER_REQUEST	Uncovered account request	Human-support hand-off.
other-001	OTHER_REQUEST	Out-of-scope creative request	Human-support hand-off.
edge-001	BUG_REPORT	Very short 'It is broken.'	Treat as likely bug; collect fields.
security-001	OTHER_REQUEST	Prompt-injection attempt	Do not reveal internal instructions.

Required runtime evidence

- Architecture/flow screenshot showing the complete solution.
- System prompt configuration showing the routing and bug-report checklist.
- AgentCore Gateway target/tool configuration.
- Successful multi-turn bug-report chat transcript.
- Visible `[tool call] bugreports___create_bug_report` event.
- DynamoDB item showing ticketId, description, stepsToReproduce, environment, and OPEN status.
- Covered FAQ response, uncovered FAQ hand-off, and other-request hand-off.
- Generated JSONL evaluation dataset.
- Bedrock Evaluations result page and written observations

Evidence status in this PDF: the test plan and expected evidence are documented from the project package. Actual AWS console screenshots and a final evaluation score should be inserted after the project is executed in the course environment; they are not fabricated here.

6. Submission-Ready Checklist

Rubric area	Deliverable/evidence	Package status
Classification & routing	System prompt + architecture documentation	Prepared
Bug report path	Prompt + Lambda + CloudFormation + test scenarios	Prepared
Gateway tool	Gateway setup wrapper + tool name	Prepared
FAQ path	Embedded FAQ + grounded-answer tests	Prepared
Other request path	Human-support hand-off rule + tests	Prepared
Automated testing	harness-tests.json + evaluation generator	Prepared
Bedrock Evaluation	JSONL + evaluation job result + observation	Runtime evidence required

Rubric area	Deliverable/evidence	Package status
DynamoDB verification	Ticket record created by chatbot	Runtime evidence required

Professional project identity

Edris Abdella Nuure — AWS AI & ML Scholars Program / Future AWS Agent Engineer
Phone: +251905131051 / +251944676746
Email: edrisabdella178@gmail.com / engineeredrisabdella@gmail.com
LinkedIn: linkedin.com/in/edris-abdella-7aa521177

Recommended final submission package: this PDF + the complete project ZIP + the required runtime screenshots/transcript + the generated evaluation JSONL + the Bedrock Evaluations result and written observations.

7. Technical Appendix

Core project files

File	Purpose
system_prompt.txt	Main routing, grounding, tool-use, and security behavior.
online_shop_faq.md	Fictional FAQ embedded through the {{FAQ}} placeholder.
cloudformation-tool.yaml	DynamoDB, Lambda, and IAM tool-stack infrastructure.
create_bug_report.py	Lambda implementation for ticket persistence.
setup_gateway.py	AgentCore Gateway integration/setup wrapper.
create_harness.py	Managed-harness creation/update wrapper.
chat.py	Interactive multi-turn testing entry point.
harness-tests.json	Automated route and edge-case test suite.
generate-eval-dataset.py	Evaluation dataset generation entry point.
cleanup_agentcore.py	Cleanup workflow/checklist.

Operational verification commands

```
aws dynamodb scan --table-name bug-report-tool-stack-bug-reports --region us-east-1
```

A successful bug-report run should produce a real ticket record. Verify that the record's fields correspond to the customer's conversation and that the status is OPEN. Do not use a manually fabricated ticket ID as evidence.

Final quality standard

- No unsupported claims about AWS runtime execution are included in this report.
- No customer-facing answer should invent FAQ content or support policies.
- No ticket should be claimed as created unless the tool returns success.
- The final evaluation score should be reported exactly as observed in Bedrock Evaluations.
- Screenshots should clearly show the relevant AWS resource, configuration, or result and be readable at submission size.